Chapitre 2 : Classes et Objets

L'objectif de ce chapitre est d'introduire deux des notions fondamentales de la programmation orientée objet, l'**objet** et la **classe**. Cette fiche résume également les notions connexes telles que l'**encapsulation**, la gestion de la mémoire en Java, les tableaux en Java et la notion de membre de classe.

Classes et objets

L'**objet** est la notion centrale en POO. Un objet est caractérisé par :

- un état (les valeurs de ses *attributs*) ;
- un comportement (les *méthodes* pouvant lui être appliquées), des services que l'objet peut nous rendre ;
- une identité (par exemple une adresse en mémoire).

Un objet n'a de « réalité » qu'à l'exécution du programme. On accède à un objet par l'intermédiaire d'une référence (en Anglais, *handle*, ce qui signifie littéralement « poignée »), qui a beaucoup de points communs avec un pointeur C. L'instruction Java `new` sert à créer un objet. Cette instruction réserve l'espace mémoire nécessaire, initialise les attributs de l'objet créé et renvoie la référence.

Créer un objet

Prenons un exemple. Un pangolin est un objet qui peut être caractérisé de la manière suivante :

Pangolin ?
Petit mammifère d'Asie et d'Afrique ressemblant « à un artichaut à l'envers avec des pattes, prolongé d'une queue à la vue de laquelle on se prend à penser qu'en effet, le ridicule ne tue plus. » (Pierre Desproges)

- état : un nom, un nombre d'écailles, une position spatiale
- comportement : translater, crier

Pour créer un pangolin (et pouvoir l'utiliser après), il faut déclarer une référence de type `Pangolin` et utiliser l'opérateur `new` pour créer effectivement l'objet.

```
Pangolin gerard;          // déclare une référence de type Pangolin
gerard = new Pangolin(); // réserve la zone mémoire nécessaire et l'affecte à la référence précédente
```

La valeur par défaut d'une référence est la valeur `null`, qui signifie approximativement que la référence ne pointe sur rien du tout.

Voici schématiquement (et de manière très imagée) ce qui se passe dans la mémoire à l'exécution des instructions précédentes.

1. Création de la référence (1ère instruction), initialement de valeur `null`

2. Création de l'objet et affectation de cet objet à la référence.

Les classes

À quel endroit les caractéristiques des objets (ce qui définit leur état et leur comportement) sont-elles définies ? Dans les **classes**. La définition d'une classe comporte trois éléments :

- son nom ;
- la liste de ses attributs (caractérisant l'état) ;
- la liste de ses méthodes (les services que tout objet de cette classe peut nous rendre).

Voici la syntaxe de déclaration d'une classe, illustrée sur un exemple.

```
class Pangolin {
    // Ci-dessous la déclaration des attributs de la classe
    double x;
    double y;
    String name;
    int nbEcailles;

    // La déclaration des méthodes avec leur code
    void translater(double dx, double dy) {
        this.x += dx;
        this.y += dy;
    }

    void crier() {
        System.out.println("Gwwark Rhââgn Bwwikk"); // Cri du pangolin
    }
}
```

La définition d'une classe permet :

- de typer des références (une classe définit un *type de données*) ;
- de créer des objets (appelés *instances de la classe*) avec l'opérateur `new` ;
- d'accéder à l'état interne avec le `.` (par exemple `gerard.x = 2;`) ;
- d'appeler (invoquer) des méthodes avec le `.` (par exemple `gerard.crier();`).

Remarque

On peut remarquer dans le code ci-dessus la présence de la variable `this`. Cette variable contient une référence à l'objet sur lequel la méthode est invoquée. Il s'agit d'un paramètre implicite passé lorsque l'on appelle une méthode sur un objet. Par exemple, lors de l'appel `gerard.translater(0, 1);`, `this` contient une référence vers le même objet que `gerard`; c'est son état qui sera modifié par la méthode.

Types primitifs et objets en Java

En Java, il existe deux catégories de types :

- les types primitifs à la C : `int`, `float`, `char`, `double`, `boolean`, ...
- les types objets, qui sont des références (on ne manipule pas directement la valeur, mais on le fait par l'intermédiaire d'un « pointeur »).

Notez que les types primitifs s'écrivent en minuscule alors que les noms de classes commencent toujours par une majuscule (si vous suivez les conventions de codage, ce qui est fortement recommandé!). Ainsi votre classe doit être nommée `Pangolin` et non `pangolin`, et son attribut `name` est de type `String` - qui est donc une classe!

Le mécanisme de JVM est fondamental pour la portabilité, le codage des nombres étant ainsi indépendant de l'architecture des machines!

Le tableau ci-dessous résume les différents types primitifs.

	Taille en octets	**Valeur minimale (inclue)**	**Valeur maximale (inclue)**
entiers			
`byte`	1	-128	127
`short`	2	-32768	32767
`int`	4	-2147483648	2147483647
`long`	8	-9223372036854775808	9223372036854775807
caractères			
`char`	2	`'\u0000'` = 0	`'\uffff'` = 65535
flottants			
`float`	4	$-3.4028235 \times 10^{38}$	3.4028235×10^{38}
`double`	8	$-1.7976931348623157 \times 10^{308}$	$1.7976931348623157 \times 10^{308}$

Seul le type primitif `boolean` n'est pas dans le tableau. Tout ce que l'on sait de `boolean` c'est qu'il a deux valeurs : `true` et `false`. On ne peut pas l'utiliser comme un nombre (`false` n'est pas `0`, `true` n'est pas `1`) et sa taille en mémoire n'est pas spécifiée.

Une variable locale non initialisée (`int x;` ou `Pangolin gerard;`) ne peut pas être utilisée directement (par exemple `x++;` ou `gerard.crier();`). Mais vous avez de la chance, le compilateur devrait vous tenir au courant (*"The local variable xxx may not have been initialized."*)

En Java, la plupart des variables manipulées sont du second type (objet). Attention donc, on manipule des **références** ! Regardez par exemple le code suivant.

```
Pangolin gerard = new Pangolin();
Pangolin toto = gerard;
toto.translater(1, 0); // Que vaut alors gerard.x ?
```

Conventions d'écriture (alias *Coding style*)

Plusieurs règles d'écriture sont en vigueur dans la communauté des développeurs Java. On désigne ces règles sous la terminologie de *Coding style*. Voici par exemple celles portant sur les noms :

- Un nom de variable, d'attribut ou de méthode commence par une minuscule et est écrit en *Camel Case*. Exemple : `maBellevariable`

Camel Case ?
Dans une variable écrite en Camel Case, chaque mot est en minuscule et commence par une majuscule (sauf éventuellement le premier mot). Il n'y a pas de séparateur de mot.

- Un nom de classe commence par une majuscule et est écrit en Camel Case. Exemple : `MaBelleClasse`
- Un nom de constante est intégralement en lettres capitales, mots séparés pas des tirets bas. Exemple : `MA_BELLE_CONSTANTE`

Retenez aussi notamment les indentations de taille 4 et les accolades ouvrantes en fin de ligne. Regardez les exemples de ce cours, qui naturellement respectent ces règles.

Vous trouverez au lien suivant le Coding style Java. Officiellement il n'est plus maintenu par Oracle, mais est toujours en vigueur.

Attention

Il y a peu de langages dans lesquels les conventions d'écriture sont aussi respectées. Libre à vous de ne pas les suivre, mais alors, un grand malheur s'abattra sur vous et votre équipe de développement pendant sept générations.

Visibilité et encapsulation

Quel est le problème du code suivant ?

```
Pangolin gerard = new Pangolin();
gerard.nbEcailles = -1;
```

C'est assez évident : en ayant accès à tous les attributs internes de la classe Pangolin, l'utilisateur de cette classe peut mettre n'importe quel objet dans un état incohérent (par exemple un état dans lequel le nombre d'écailles est négatif, ce qui n'a pas de sens.

Une manière de résoudre ce problème est de restreindre la **visibilité** des attributs et méthodes à l'aide de modificateurs. C'est l'*encapsulation*. En Java, il existe quatre niveaux de visibilité pour les attributs et méthodes, dont les effets sont résumés dans le tableau ci-dessous (les notions de sous-classes et de paquetages seront précisés plus tard).

	Classes des autres paquetages	**Sous-classes**	**Autres classes, même paquetage**	**Intérieur de la classe**
`public` (+)	✔	✔	✔	✔
`protected` (#)	✗	✔	✔	✔
défaut ()	✗	✗	✔	✔
`private` (-)	✗	✗	✗	✔

- ✔ : accès possible (lecture et écriture)
- ✗ : accès interdit (lecture et écriture)

Reprenons la classe Pangolin développée ci-dessus, et précisons les visibilités pour chacun des attributs et des méthodes.

```
class Pangolin {
    // Ci-dessous la déclaration des attributs de la classe
    private double x;
    private double y;
    private String name;
    private int nbEcailles;

    // La déclaration des méthodes avec leur code
    public void tranlater(double dx, double dy) {
        x += dx;
        y += dy;
    }

    public void crier() {
        System.out.println("Gwwark Rhââgn Bwwikk"); // Cri du pangolin
    }
}
```

Avec une telle déclaration, le code suivant :

```
Pangolin gerard = new Pangolin();
gerard.nbEcailles = -1;
```

est incorrect s'il est écrit dans une autre classe que Pangolin. L'attribut `nbEcailles` (comme tous les autres attributs) est invisible de l'extérieur de la classe.

Il existe un principe fondamental en Programmation Orientée Objet : le ***principe d'encapsulation***. Ce principe stipule que la représentation de l'état interne d'une classe d'objets n'est connue et accessible que de cette classe, et qu'en particulier aucune autre classe ne peut modifier directement l'état interne d'un objet. Cela se traduit de la manière (simplifiée) suivante.

Principe d'encapsulation (simplifié)

Tout attribut d'une classe doit être déclaré privé.

Accesseurs, mutateurs

Il existe cependant de nombreuses situations où l'on a besoin d'accéder à des valeurs d'attribut d'un objet, soit pour les consulter, soit pour les modifier. Dans ce cas-là, une solution est d'utiliser des méthodes particulières appelées accesseurs (*accessors*) et mutateurs (*modifiers*). Exemple sur Pangolin.

```
class Pangolin {
    // Ici le reste de classe Pangolin (comme précédemment)
    // [...]
    public int getNbEcailles() {
        return this.nbEcailles;
    }

    public void setNbEcailles(int nb) {
        if (nb <= 0) {
            throw new IllegalArgumentException("Le nombre d'écailles doit
être strictement positif !");
        }
        this.nbEcailles = nb;
    }
}
```

L'avantage d'utiliser des accesseurs et mutateurs est que l'on peut finement contrôler l'accès à l'état interne d'un objet, et toujours garantir son intégrité.

Il est important de **ne pas écrire systématiquement** les set/get, mais seulement lorsque c'est nécessaire. Une classe a très souvent des attributs privés auxquels on ne doit pas accéder et/ou qui ne doivent pas être modifiés depuis l'extérieur.

Construire, détruire

Construire

La visibilité permet de résoudre le problème d'incohérence lié à un accès non contrôlé aux attributs. En revanche, elle ne permet pas de résoudre les

problèmes potentiels d'incohérence liés à l'initialisation des attributs aux valeurs par défaut. Considérons par exemple le code suivant.

```
Pangolin gerard = new Pangolin();
gerard.setNbEcailles(1542);
```

Entre les lignes 1 et 2, notre pangolin est dans un état incohérent car son nombre d'écailles est nul. Pour éviter ce problème, il faut rendre *atomique* la création d'un objet (la réservation de la zone mémoire) et l'initialisation de ses attributs. C'est exactement le rôle des *constructeurs*.

Un constructeur est une « méthode » spécifique d'une classe, portant le nom de cette classe, *sans type de retour*, et qui exécute du code au moment de la création de l'objet. Voici un exemple.

```
class Pangolin {
    // Ici le reste de classe Pangolin (comme précédemment)
    // [...]
    public Pangolin(String nom, double xInit, double yInit, int nbEcailles)
{
        this.nom = nom;
        this.x = xInit;
        this.y = yInit;
        this.setNbEcailles(nbEcailles);
    }
}
```

Le constructeur est appelé à la création de l'objet, c'est-à-dire lors de l'utilisation de l'opérateur `new`. Ainsi, par exemple, le code suivant construit un pangolin en appelant le constructeur déclaré ci-dessus, c'est-à-dire en initialisant les attributs aux valeurs passées en paramètre.

```
Pangolin gerard = new Pangolin("Gérard", 0, 0, 1542);
```

Ainsi, plus de problème d'incohérence à l'initialisation.

Une classe peut contenir plusieurs constructeurs. Ces constructeurs peuvent même faire appel l'un à l'autre. Exemple :

```
class Pangolin {
    // Ici le reste de classe Pangolin (comme précédemment)
    // [...]
    public Pangolin(String nom, int nbEcailles) {
        this(nom, 0, 0, nbEcailles); // Appel au constructeur à quatre
paramètres
    }
}
```

Valeurs par défaut

Lors de l'appel à `new`, plusieurs opérations sont en fait réalisées:

1. allocation de la mémoire pour l'objet créé ;
2. initialisation de tous les attributs, avec une valeur par défaut ou celle spécifiée ;
3. exécution du constructeur (s'il y en a plusieurs, choisi selon les types des arguments).

Il est donc possible de spécifier la valeur initiale d'un attribut directement lors de sa déclaration :

```
class Toto {
    private int a = 17;
    private String s = new String("zut");
    // [...]
}
```

Si aucune valeur n'est spécifiée, les entiers seront initialisés à `0`, les flottants à `0.0`, les booléens à `false` et toutes les références à `null`. Le constructeur n'est exécuté qu'*après* ces initialisations.

Constructeur par défaut

Lorsqu'il n'y a aucun constructeur explicitement défini dans une classe, Java en synthétise un, qui ne prend aucun paramètre et... ne fait rien. Toutes les attributs sont donc initialisés à leur valeur par défaut. C'est ce constructeur (appelé *constructeur par défaut*) qui était appelé dans les tout premiers programmes de cette fiche lorsque l'on faisait un `new Pangolin()`.

Attention, dès qu'un constructeur est explicitement défini, ce constructeur par défaut disparaît (et dans ce cas-là, l'instruction `new Pangolin()` devient invalide s'il n'existe pas de constructeur sans paramètre dans la classe Pangolin).

Détruire ?

En Java, on ne peut jamais détruire explicitement un objet, donc on ne peut pas libérer explicitement une zone de la mémoire occupée par un objet. Il arrive malgré tout que des objets créés deviennent inutiles au cours du fonctionnement du programme. Comment éviter la saturation de la mémoire, dans ce cas ?

En fait, c'est le rôle du *ramasse-miettes* (*garbage collector*) de la machine virtuelle Java. Ce ramasse-miettes est lancé de manière périodique par la JVM et se charge de repérer les objets n'ayant plus aucune référence pointant sur eux. Ces objets sont par définition inutiles, car inaccessibles par le programme. La zone mémoire qu'ils occupent peut donc être marquée libre.

Les tableaux en Java

En Java, un tableau désigne un ensemble d'objets de même type, à accès direct par indice, et de taille fixe (non modifiable). Un tableau a toutes les caractéristiques d'un objet (en particulier, un tableau se manipule par l'intermédiaire d'une référence). La manipulation de tableaux passe par une syntaxe spécifique inspirée du C : les crochets [].

Voici la syntaxe, via quelques exemples :

```java
// 1. Un premier tableau d'entiers
int[] tab;          // déclaration, tab n'est pas encore initialisé
tab = new int[5];   // allocation;
                    // => les indices vont de 0 à tab.length - 1, ici 4
                    //  => les valeurs sont initialisées à 0
tab[0] = 2;         // accès via l'operateur []

// 2. tableau 1D d'objets
int n = 42;
Pangolin[] pangolins = new Pangolin[n]; // attention, ne contient que des références null!

for (int i = 0; i < pangolins.length; i++) {
    pangolins[i] = new Pangolin(...);  // allocation objet par objet
}

// autre parcours, de type "for each"
// (pour appliquer un traitement à *tous* les éléments)
for (Pangolin p : pangolins) {
    System.out.println(p.toString());
}

// 3. Tableau 2D
int[][] tab2D;
tab2D = new int[5][]; // tab de 5 références null
for (int i = 0; i < tab2D.length; i++) {
    // allocation case par case. Dimensions par forcément identiques
    tab2D[i] = new int[i + 1];
}
// et maintenant, tout simplement :
tab2D[1][0] = 45;
```

Remarques

Un tableau étant paramétré par n'importe quel type Java, on peut bien entendu créer des tableaux de tableaux, ce qui permet de simuler par exemple des matrices (par exemple : `Pangolin[][] mat = new Pangolin[10][10]; // 100 pangolins !`).

On peut accéder à la longueur d'un tableau grâce au pseudo-attribut `length` (par exemple : `int l = tab1.length`)

Les indices sont numérotés de 0 à `length - 1`. Un débordement d'indice provoque une exception de type `ArrayIndexOutOfBoundsException`.

Les types énumérés en Java

Un type énuméré est une classe particulière qui ne possède qu'un nombre restreint d'objets (qui sont en fait des constantes). Voici un exemple minimal :

```java
enum Couleur {
  ROUGE, VERT, BLEU;
}
```

Dans l'exemple ci-dessus, `Couleur` est donc un type qui ne contient que trois valeurs possibles : `ROUGE`, `VERT` et `BLEU`.

On peut donc utiliser cette énumération pour typer n'importe quelle variable, ce qui peut être utile partout où l'on a besoin de raisonner avec

des constantes particulières (utile par exemple pour faire un `switch` ou utiliser des conditionnelles s'appuyant sur la valeur particulière d'une certaine couleur).

On peut également itérer à travers toutes les valeurs d'un type énuméré en utilisant la méthode `values()` (pour avoir une idée de la syntaxe, jetez un œil à la section sur les itérateurs, dans la fiche sur les collections).

Bon, tout cela est très intéressant, mais notre type énuméré `Couleur` est assez basique pour le moment. Ça tombe bien, on peut le complexifier un peu en lui ajoutant des attributs et des méthodes, à la manière d'une classe. Jugez plutôt :

```
enum Couleur {
    ROUGE(255, 0, 0), VERT(0, 255, 0), BLEU(0, 0, 255);

    private int rouge, vert, bleu;

    Couleur (int rouge, int vert, int bleu) {
        this.rouge = rouge;
        this.vert = vert;
        this.bleu = bleu;
    }

    public String getHtmlCode() {
        return "#" + String.format("%02x", this.rouge)
            + String.format("%02x", this.vert)
            + String.format("%02x", this.bleu);
    }
}
```

Ainsi, par exemple, on pourra écrire `Couleur c = ROUGE; System.out.println(c.getHtmlCode());` pour récupérer le code HTML associé à la couleur rouge.

Voilà, vous savez tout.

Éléments particuliers : point d'entrée, affichage, comparaison, ...

Méthode principale en Java

En Java, pour lancer un programme, il faut définir une méthode principale dans une classe, en utilisant la syntaxe suivante :

```
  // Le nom de la classe est à votre convenance.
  // L'usage est d'appeler TestXXX une classe qui contient le programme
principal
  public class TestPangolin {
      public static void main(String[] args) {
          // Le tableau args, de taille  args.length, contient les
arguments du programme
          // Ici définissez les instructions de votre programme principal
      }
  }
```

La classe doit être *publique*, via le mot-clé `public` devant `class`! Cette notion sera vue dans la fiche sur les paquetages.

Il convient ensuite de compiler ce fichier et d'interpréter le code binaire correspondant pour exécuter le programme: `java TestPangolin`. Tout ceci est expliqué en détails ici: compilation et exécution.

Dans un même projet, plusieurs classes peuvent définir une méthode `main`. Le point d'entrée de l'exécution dépend simplement de la manière dont elle est lancée: `java Classe1` ou `java Classe2` invoque la méthode `main` de la classe spécifiée.

La méthode toString()

Dans de nombreux cas, il peut être utile d'avoir une description textuelle d'un objet (pour l'afficher à l'écran par exemple). Pour cela, l'usage est de définir une méthode s'appelant `tostring()` et renvoyant une chaîne de caractères. Exemple sur Pangolin :

```
  class Pangolin {
      // Ici le reste de classe Pangolin (comme précédemment)
      // [...]

      @Override
      public String toString() {
          return "Le Pangolin " + this.nom + "(" + this.nbEcailles + "
écailles)"
      }
  }
```

Un intérêt majeur de cette méthode est que la JVM l'appelle implicitement à chaque endroit où elle a besoin d'une représentation textuelle d'un objet. Ainsi, par exemple, le code `System.out.println(gerard);` devrait être invalide car la méthode `System.out.println` attend une chaîne de caractères en paramètre. Cependant, elle est valide en pratique, car la JVM ajoute implicitement l'appel à la méthode `tostring()` sans que l'on n'ait rien demandé: `System.out.println(gerard.toString());`.

C'est un peu magique non? Comment le compilateur sait que la méthode `toString()` existe dans `Pangolin`? D'ailleurs existe-t-elle?

Pour le savoir, restez avec nous pour les prochains cours !

Comparaison de référence, comparaison sémantique

Avancé !

N'ayez pas peur nous reviendrons dessus, mais retenez que l'opérateur `==` ne compare que des références, pas des objets. Danger !

Il existe un opérateur de comparaison en Java : `==`. Cet opérateur renvoie `true` uniquement lorsque les deux références comparées pointent sur le même objet en mémoire.

```
Pangolin gerard = new Pangolin("Gérard", 0, 0, 1452);
Pangolin gerard2 = new Pangolin("Gérard", 0, 0, 1452);
System.out.println(gerard == gerard2); // Écrit "false"
gerard2 = gerard;
System.out.println(gerard == gerard2); // Écrit "true"
```

Souvent, ce n'est pas ce que l'on cherche lorsque l'on veut comparer deux objets. Prenons par exemple deux chaînes de caractères (contrairement à ce que l'on pourrait penser, le type chaînes de caractères n'est pas un type primitif en Java, mais un type objet `String`). Le recours à l'opérateur `==` ne donnera pas le résultat attendu :

```
String s1 = new String("abc");
String s2 = new String("abc");
System.out.println(s1 == s2); // Écrit "false"
```

C'est complètement normal. Même si `s1` et `s2` contiennent la même chaîne de caractères, il s'agit bien de deux objets différents (dont le tableau de caractères dont ils sont constitués est le même). Si l'on veut faire une comparaison *sémantique* des chaînes de caractères (c'est-à-dire comparer les caractères dont elles sont constituées), il faut utiliser la méthode `equals()`.

```
String s1 = new String("abc");
String s2 = new String("abc");
System.out.println(s1.equals(s2)); // Écrit "true"
```

Le fonctionnement de cette comparaison sémantique n'est pas magique. Java ne peut pas deviner tout seul ce que veut dire le fait d'être égal pour des objets d'une certaine classe. Cette méthode `equals()` doit donc être écrite explicitement pour chaque classe pour laquelle on a besoin d'une comparaison sémantique (elle l'est dans la classe `String()` par exemple). Si l'on veut par exemple spécifier que deux pangolins sont égaux si et seulement s'ils ont le même nom, il faudra définir cette méthode dans la classe Pangolin, de la manière suivante.

```
class Pangolin {
    // Ici le reste de classe Pangolin (comme précédemment)
    // [...]

    @Override
    public boolean equals(Object other) {
        if (other instanceof Pangolin) {
            return ((Pangolin) other).nom.equals(this.nom);
        }
        return false;
    }
}
```

Remarque

On peut remarquer la présence de l'opérateur Java `instanceof` qui permet de tester si un objet est l'instance d'une certaine classe. Attention au type du paramètre de `equals` qui est bien `Object`. Quant à l'annotation `@Override`, son utilisation sera détaillée plus tard.

Remarque 2

En fait, la gestion des objets `String` est un peu plus compliquée que ce qui a été exposé plus haut et peut résulter en un comportement surprenant de l'opérateur d'égalité. Si vous voulez en savoir un peu plus (et que vous comprenez les risques que vous prenez en accédant à tant de savoir nouveau et avancé), cliquez sur le bouton ci-dessous :

String pooling (info. avancée)
Vous l'aurez voulu. Considérons par exemple le code suivant :

```
String s1 = "abc";
String s2 = "abc";
System.out.println(s1 == s2);
```

Dans ce code, l'égalité de la troisième ligne renverra `true`. Pourquoi ? Parce que la JVM utilise ce que l'on appelle l'*internalisation de String* (*String pooling*) : lorsque les chaînes de caractères sont créées, tout se passe ici comme si Java ne créait pas de nouvel objet et qu'il se contentait de faire une référence à un objet déjà créé dans une certaine zone de la mémoire, et qui contiendrait directement la chaîne `"abc"`. Cela explique pourquoi l'égalité renvoie vrai.

Attention toutefois : si vous utilisez l'opérateur `new` pour créer votre chaîne de caractères, comme dans `String s1 = new String("abc");`, Java n'internalise plus automatiquement la chaîne de caractères et crée bien un nouvel objet. Il faudra appeler explicitement la méthode `intern()` (par exemple `s1.intern()`) pour qu'il le fasse.

Moralité : si vous n'êtes pas sûr de ce que vous faites, la manière la plus simple de vous en tirer est encore d'utiliser systématiquement la méthode `equals()` pour comparer vos chaînes de caractères.

Attributs et méthodes de classe

Les attributs et méthodes dont nous avons parlé jusqu'ici sont relatifs à un objet particulier (une instance). Ainsi, par exemple, la valeur de l'attribut `nbEcailles` dépend de l'objet duquel on parle, et pourra être différente selon qu'il s'agisse de `gerard` ou `toto`. De même, lorsque l'on appelle la méthode `translater()`, c'est bien un pangolin particulier que l'on va translater. On parle donc d'*attributs d'instance* et de *méthodes d'instance*.

Il n'en est pas de même pour tous les attributs et méthodes. Ainsi, par exemple, imaginons un attribut, dans la classe `Pangolin`, qui stocke le nombre d'instances de pangolins créés jusqu'ici. La valeur de cet attribut ne varie évidemment pas selon les objets. Elle est au contraire commune à tous les objets de la classe. L'attribut n'est pas relatif aux objets, mais à la classe.

Un tel attribut est dit *de classe*, ou encore *statique*. Un attribut de classe est déclarée avec le mot-clef `static` et on peut y accéder avec le nom de la classe suivi de la notation usuelle .. Exemple avec la classe Pangolin :

```
class Pangolin {
    // Ici le reste de classe Pangolin (comme précédemment)
    // [...]

    // Un attribut de classe, partagé par tous les pangolins
    private static int nbPangolins = 0;

    // Une méthode de classe, indépendante de toute instance particulière
    public static int getNbPangolins() {
        return nbPangolins; // ou return Pangolin.nbPangolin ;
    }

    public Pangolin(String nom, double xInit, double yInit, int nbEcailles)
{
        nbPangolins++; // Un Pangolin de plus a été créé ! On peut aussi
écrire Pangolin.nbPangolins ++ ;
        this.nom = nom;
        this.x = xInit;
        this.y = yInit;
        this.setNbEcailles(nbEcailles);
    }
}
```

L'accès à un attribut de classe et l'invocation d'une méthode de classe peuvent se faire, comme d'habitude, au moyen d'une référence vers une instance. Mais, puisqu'ils ne sont liés à aucune instance particulière, il est aussi possible d'utiliser le nom de la classe, et non pas un objet :

```
    Pangolin gerard = new Pangolin() ;
    // récupère le nb de pangolins crées jusque-là
    System.out.prinln( gerard.getNbPangolins() ) ; // Bon, ça marche...
Affiche 1.

    // Mais on peut aussi accéder à un membre de classe directement avec le
nom de la classe.
    // Et c'est d'ailleurs ce qu'on fait en général avec les membres de
classe !
    System.out.prinln( Pangolin.getNbPangolins() ); // Affiche 1 aussi.
```

Quizz : Que se passe-t-il si vous utilisez la référence `this` dans une méthode de classe?

Le compilateur vous jette avec une erreur du type `error: non-static variable this cannot be referenced from a static context`. Et il a bien raison d'ailleurs. Vu que l'on est dans une méthode de classe, une telle méthode est appelée indépendamment de tout objet de cette classe. Aucun objet n'est donc passé en paramètre implicite de cette méthode, et donc la variable `this` n'existe pas dans ce contexte.

Remarques

Le mot clé `static` est très utilisé pour déclarer des attributs constants : comme la valeur d'une constante est la même pour toutes les instances, on en fait un attribut de classe. Exemple :

```
class Truc {
    public static final int UNE_CONSTANTE = 5;
}
```

D'autres usages sont possibles.

Parmi les méthodes statiques les plus usuelles, on trouve l'ensemble des opérateurs mathématiques qui par définition ne sont pas relatifs à un objet particulier (exemple : `Math.sin(Math.PI)`), les entrées / sorties standard et d'erreur (`System.in`, `System.out` et `System.err`) ou encore la méthode principale du programme (`main`).

Vous remarquerez qu'il n'est pas nécessaire d'instancier la classe (créer un objet de cette classe) pour utiliser une de ses méthodes statiques. Ce n'est d'ailleurs pas possible dans bien des cas, par exemple pour `Math`.